

535514: Reinforcement Learning

Lecture 7 — Deterministic PG

Ping-Chun Hsieh

April 1, 2025

Stochastic PG vs Deterministic PG

- ▶ So far, we have focused on stochastic policy gradient
- ▶ **Question:** How to extend policy gradient framework to parametrized deterministic policies?
- ▶ **Question:** What's the benefit of learning deterministic policies?

Stochastic PG:
$$\nabla_{\theta} V^{\pi_{\theta}}(\mu) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^{\pi_{\theta}}} \mathbb{E}_{a \sim \pi_{\theta}(\cdot | s)} \left[Q^{\pi_{\theta}}(s, a) \nabla_{\theta} \log \pi_{\theta}(a | s) \right]$$

*integration over both state and action spaces
⇒ require more samples to estimate the gradient*

Why is Deterministic PG Challenging?

- **Recall:** Stochastic policy gradient (P1), $a_t \sim \pi_\theta(\cdot | s_t)$

$$\begin{aligned}\nabla_\theta V^{\pi_\theta}(\mu) &= \sum_\tau R(\tau) \nabla_\theta P_\mu^{\pi_\theta}(\tau) \\ &= \sum_\tau R(\tau) \left(P_\mu^{\pi_\theta}(\tau) \cdot \nabla_\theta \log(\mu(s_0) \pi_\theta(a_0 | s_0) P(s_1 | s_0, a_0) \pi_\theta(a_1 | s_1) \cdots) \right) \\ &= \sum_\tau R(\tau) \left(P_\mu^{\pi_\theta}(\tau) \cdot \sum_{t=0}^{\infty} \nabla_\theta \log \pi(a_t | s_t) \right)\end{aligned}$$

- **Question:** How about deterministic policies $a_t = \pi_\theta(s_t)$?

$$\begin{aligned}\nabla_\theta V^{\pi_\theta}(\mu) &= \sum_\tau R(\tau) \nabla_\theta P_\mu^{\pi_\theta}(\tau) \\ &= \sum_\tau R(\tau) \left(P_\mu^{\pi_\theta}(\tau) \cdot \nabla_\theta \log(\mu(s_0) P(s_1 | s_0, \pi_\theta(s_0)) P(s_2 | s_1, \pi_\theta(s_1)) \cdots) \right) \\ &= \sum_\tau R(\tau) \left(P_\mu^{\pi_\theta}(\tau) \cdot \sum_{t=0}^{\infty} \nabla_\theta \log P(s_{t+1} | s_t, \pi_\theta(s_t)) \right)\end{aligned}$$

How to calculate this gradient?

Challenge: It seems we need to know the model P to find the gradient

Despite this, the deterministic PG indeed exists
and enjoys a simple form!

Deterministic Policy Gradient (DPG)

- ▶ Consider continuous actions and deterministic policy: $a = \pi_\theta(s)$
- ▶ **Assumptions:** $\nabla_a Q(s, a)$, $\nabla_a P(s' | s, a)$, $\nabla_\theta \pi_\theta(s)$, $\nabla_a R(s, a)$ exist

- ▶ **Deterministic Policy Gradient Theorem:**

$$\nabla_\theta V^{\pi_\theta}(\mu) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d_\mu^{\pi_\theta}} \left[\nabla_\theta \pi_\theta(s) \nabla_a Q^{\pi_\theta}(s, a) \Big|_{a=\pi_\theta(s)} \right]$$

- ▶ **Comparison:** Stochastic policy gradient

(P3) Q-value and discounted state visitation:

$$\nabla_\theta V^{\pi_\theta}(\mu) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d^{\pi_\theta}} \mathbb{E}_{a \sim \pi_\theta(\cdot | s)} \left[Q^{\pi_\theta}(s, a) \nabla_\theta \log \pi_\theta(a | s) \right]$$

Connecting Stochastic PG and Deterministic PG

Deterministic PG: $\nabla_{\theta} V^{\pi_{\theta}}(\mu) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d_{\mu}^{\pi_{\theta}}} \left[\nabla_{\theta} \pi_{\theta}(s) \nabla_a Q^{\pi_{\theta}}(s, a) |_{a=\pi_{\theta}(s)} \right]$

Stochastic PG: $\nabla_{\theta} V^{\pi_{\theta}}(\mu) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d^{\pi_{\theta}}} \mathbb{E}_{a \sim \pi_{\theta}(\cdot|s)} \left[Q^{\pi_{\theta}}(s, a) \nabla_{\theta} \log \pi_{\theta}(a | s) \right]$

-
- ▶ **Question:** Any connection between the two PGs?
 - ▶ Consider a stochastic policy: $\nu_{\sigma}(\pi_{\theta}, a)$ with $\nu_0(\pi_{\theta}(s), a) \equiv \pi_{\theta}(s)$

- ▶ **Deterministic PG is the Limiting Case of Stochastic PG:**

Under mild technical conditions on ν_{σ} (differentiability, bounded gradient, translation invariance), we have

$$\lim_{\sigma \downarrow 0} \nabla_{\theta} V^{\nu_{\sigma}(\pi_{\theta})}(\mu) = \nabla_{\theta} V^{\pi_{\theta}}(\mu)$$

Next Step: Let's apply DPG to design algorithms

(On-Policy) Deterministic Actor-Critic Algorithm

Deterministic PG: $\nabla_{\theta} V^{\pi_{\theta}}(\mu) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d_{\mu}^{\pi_{\theta}}} \left[\nabla_{\theta} \pi_{\theta}(s) \nabla_a Q^{\pi_{\theta}}(s, a) |_{a=\pi_{\theta}(s)} \right]$

- ▶ **Deterministic Actor-Critic (with Value Function Approximation):**
 - ▶ **Critic:** estimate $Q_w \approx Q^{\pi_{\theta}}$ by TD(0) bootstrapping
 - ▶ **Actor:** updates policy parameters θ by deterministic policy gradient

Step 1: Initialize θ_0 , w_0 and step sizes α_{θ} , α_w

Step 2: Sample a trajectory $\tau = (s_0, a_0, r_1, \dots) \sim P_{\mu}^{\pi_{\theta}}$

For each step of the current trajectory $t = 0, 1, 2, \dots$

$$\Delta w_k \leftarrow \Delta w_k + \alpha_w (r_t + \gamma Q_{w_k}(s_{t+1}, a_{t+1}) - Q_{w_k}(s_t, a_t)) \nabla_w Q_w(s_t, a_t) |_{w=w_k}$$

$$\Delta \theta_k \leftarrow \Delta \theta_k + \alpha_{\theta} \gamma^t \left(\nabla_{\theta} \pi_{\theta}(s_t) \nabla_a Q_w(s_t, a) |_{a=\pi_{\theta}(s_t)} \right)$$

$$\theta_{k+1} \leftarrow \theta_k + \Delta \theta_k, w_{k+1} \leftarrow w_k + \Delta w_k$$

$= \nabla_{\theta} Q_w(s_t, \pi_{\theta}(s_t)) |_{\theta=\theta_k}$

A Quick Remark on DPG Expression

- ▶ In Deterministic Actor-Critic:

$$\Delta\theta_k \leftarrow \Delta\theta_k + \alpha_\theta \gamma^t \left(\underbrace{\nabla_\theta \pi_\theta(s_t) \nabla_a Q_w(s_t, a)|_{a=\pi_\theta(s_t)}}_{= \nabla_\theta Q_w(s_t, \pi_\theta(s_t))|_{\theta=\theta_k}} \right)$$

- ▶ In the original DPG expression:

$$\nabla_\theta V^{\pi_\theta}(\mu) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d_\mu^{\pi_\theta}} \left[\underbrace{\nabla_\theta \pi_\theta(s) \nabla_a Q^{\pi_\theta}(s, a)|_{a=\pi_\theta(s)}}_{= \nabla_\theta Q^{\pi_\theta}(s, \pi_\theta(s))|_{\theta=\theta_k} ??} \right]$$

- ▶ **Question:** Any issue with deterministic policies?

Insufficient exploration

- ▶ **Question:** Is it possible to learn π but act under another policy β ?

Off-policy learning!

Off-Policy Learning with Deterministic Policy Gradients

On-Policy vs Off-Policy

- ▶ On-policy:

Learned policy = Policy used to interact with the environment

- ▶ Off-policy:

Learned policy $\neq$ Policy used to interact with the environment

Called “behavior policy”



Kazami Hayato
(learning agent)



Asurada
(policy for interaction)

Off-Policy Learning

Off-policy learning

1. Learn a target policy $\pi_{\theta}(a | s)$ and compute $V^{\pi_{\theta}}(s)$ or $Q^{\pi_{\theta}}(s, a)$
2. In the meantime, follow a behavior policy $\beta(a | s)$

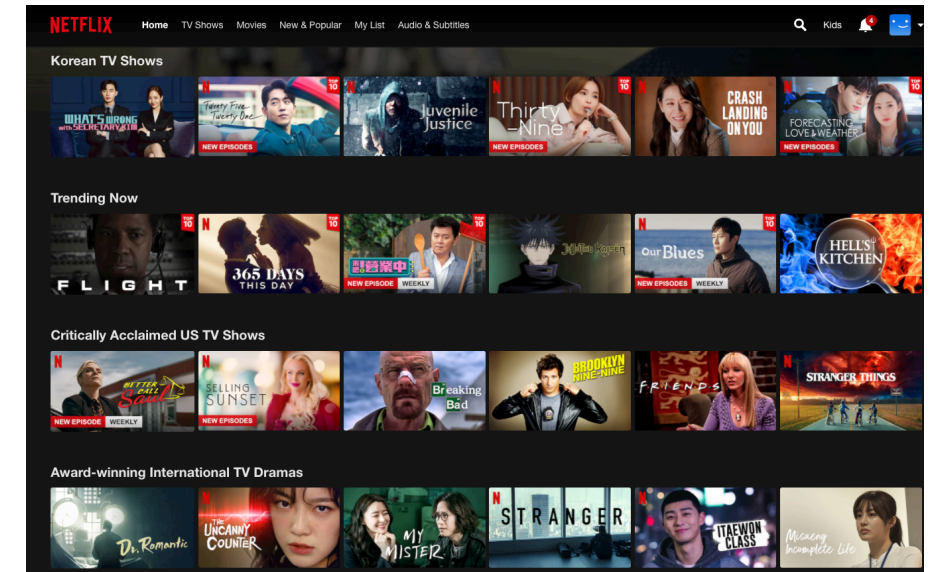
$$\{s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_{T-1}, a_{T-1}, r_T\} \sim \beta$$

► Why is off-policy learning useful?

1. Learn from observing humans or other agents
2. Reuse experience generated from old policies $\pi_1, \pi_2, \dots, \pi_{k-1}$
3. Learn about optimal policy while following an exploratory policy
4. Learn about multiple policies while following one policy

Off-Policy Learning is Essential in Many “Real-World” Problems

- Recommender Systems
 - ➔ Deploy a safe policy for collecting user data without losing user’s interest
 - ➔ Learn a better policy from these data
- Robot Control
 - ➔ Deploy a safe policy for collecting robot data without hurting the machine
 - ➔ Learn a good policy from these data



What are the behavior policies in the above applications?

Deterministic PG in Off-Policy Learning

(On-policy) DPG:
$$\nabla_{\theta} V^{\pi_{\theta}}(\mu) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d_{\mu}^{\pi_{\theta}}} \left[\nabla_{\theta} \pi_{\theta}(s) \nabla_a Q^{\pi_{\theta}}(s, a) \big|_{a=\pi_{\theta}(s)} \right]$$

- ▶ **Question:** Does the deterministic PG remain the same in off-policy learning?
Nope! (state visitation distribution shall change)

-
- ▶ Consider a new objective for off-policy learning: (Why reasonable?)

$$J_{\beta}(\pi_{\theta}) := \sum_s d_{\mu}^{\beta}(s) V^{\pi_{\theta}}(s)$$

- ▶ “Off-policy” deterministic PG: Let’s use $\nabla_{\theta} J_{\beta}(\pi_{\theta})$!

Off-Policy Deterministic PG

- ▶ **Off-Policy Deterministic Policy Gradient:**

$$\nabla_{\theta} J_{\beta}(\pi_{\theta}) \approx \mathbb{E}_{s \sim d_{\mu}^{\beta}} \left[\nabla_{\theta} \pi_{\theta}(s) \nabla_a Q^{\pi_{\theta}}(s, a) |_{a=\pi_{\theta}(s)} \right]$$

- ▶ **Question:** Is the above easy to operate with?

- ▶ **Derivation:**

$$\begin{aligned} \nabla_{\theta} J_{\beta}(\pi_{\theta}) &= \nabla_{\theta} \left(\sum_s d_{\mu}^{\beta}(s) V^{\pi_{\theta}}(s) \right) \\ &= \nabla_{\theta} \left(\sum_s d_{\mu}^{\beta}(s) Q^{\pi_{\theta}}(s, \pi_{\theta}(s)) \right) \\ &\quad \downarrow \text{The DPG paper (Silver, ICML 2014) dropped a term } \nabla_{\theta} Q^{\pi_{\theta}}(s, a) \\ &\approx \sum_s d_{\mu}^{\beta}(s) \left(\nabla_{\theta} \pi_{\theta}(s) \nabla_a Q^{\pi_{\theta}}(s, a) |_{a=\pi_{\theta}(s)} \right) \end{aligned}$$

Why is $\nabla_{\theta} Q^{\pi_{\theta}}(s, a)$ Difficult to Evaluate?

- Recall from the expression of deterministic PG:

$$\begin{aligned}\nabla_{\theta} V^{\pi_{\theta}}(s) &= \frac{1}{1 - \gamma_{\infty}} \mathbb{E}_{s' \sim d_{\mu}^{\pi_{\theta}}} \left[\nabla_{\theta} \pi_{\theta}(s') \nabla_a Q^{\pi_{\theta}}(s', a) \Big|_{a=\pi_{\theta}(s')} \right] \\ &= \sum_{s'} \sum_{t=0}^{\infty} \gamma^t P(s \rightarrow s', t, \pi_{\theta}) \nabla_{\theta} \pi_{\theta}(s') \nabla_a Q^{\pi_{\theta}}(s', a) \Big|_{a=\pi_{\theta}(s')}\end{aligned}$$

Accordingly, we have

$$\begin{aligned}\nabla_{\theta} Q^{\pi_{\theta}}(s, a) &= \nabla_{\theta} \left(R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^{\pi_{\theta}}(s') \right) \\ &= \gamma \sum_{s'} P(s' | s, a) \nabla_{\theta} V^{\pi_{\theta}}(s') \\ &= \gamma \sum_{s'} P(s' | s, a) \sum_{s''} \sum_{t=0}^{\infty} \gamma^t P(s' \rightarrow s'', t, \pi_{\theta}) \nabla_{\theta} \pi_{\theta}(s'') \nabla_a Q^{\pi_{\theta}}(s'', a) \Big|_{a=\pi_{\theta}(s'')}\end{aligned}$$

hard to evaluate in off-policy learning (why?)

Off-Policy Deterministic Actor-Critic (OPDAC) Algorithm

- ▶ Off-Policy Deterministic Actor-Critic (OPDAC):
 - ▶ **Critic**: estimate $Q_w \approx Q^{\pi_\theta}$ by TD bootstrapping
 - ▶ **Actor**: updates policy parameters θ by off-policy deterministic PG

Step 1: Initialize θ_0 , w_0 and step sizes α_θ , α_w

Step 2: Sample a trajectory $\tau = (s_0, a_0, r_1, \dots) \sim P_\mu^\beta$

For each step of the current trajectory $t = 0, 1, 2, \dots$

$$\Delta w_k \leftarrow \Delta w_k + \alpha_w (r_t + \gamma Q_{w_k}(s_{t+1}, \pi_\theta(s_{t+1})) - Q_{w_k}(s_t, a_t)) \nabla_w Q_w(s_t, a_t)|_{w=w_k}$$

$$\Delta \theta_k \leftarrow \Delta \theta_k + \alpha_\theta \gamma^t \left(\nabla_\theta \pi_\theta(s_t) \nabla_a Q_{w_k}(s_t, a)|_{a=\pi_\theta(s_t)} \right)$$

$$\theta_{k+1} \leftarrow \theta_k + \Delta \theta_k, w_{k+1} \leftarrow w_k + \Delta w_k \quad \rightarrow \quad = \nabla_\theta Q_{w_k}(s_t, \pi_\theta(s_t))|_{\theta=\theta_k}$$

- ▶ **Question**: Can you identify differences between OPDAC and DAC?

Deep Deterministic Policy Gradient (DDPG)

(= OPDAC with Deep Neural Nets)

What is DDPG?

- ▶ **DDPG**: Combine OPDAC with NN nonlinear VFA
 - ▶ **Off-policy**: Exploration
 - ▶ **Nonlinear VFA**: Convergence issue
- ▶ To tackle the above issues, DDPG applies several techniques:
 - (T1) Experience replay (for data-efficient off-policy learning)
 - (T2) Ornstein-Uhlenbeck process for exploration (optional)
 - (T3) Target networks

(T1) Experience Replay

- ▶ **Main idea:**

1. Store the previous experiences (s, a, s', r) into a buffer
2. Sample a mini-batch from the buffer at each step
(similar to mini-batch SGD in supervised learning)

- ▶ **Purposes:**

1. **Better estimate of DPG:** Break correlations between successive steps in a trajectory (“more stable learning”, as stated in many papers)
2. **Better data efficiency:** Fewer interactions with environment needed for convergence

(T2) Ornstein-Uhlenbeck Process for Exploration

- ▶ Issue with Gaussian noise exploration $a_t = \pi_\theta(s_t) + N(0, \sigma^2)$?



-
- ▶ **Ornstein-Uhlenbeck (OU) process**: Similar to Gaussian policies, but with **temporal correlation**

$$dx_t = \theta(\mu - x_t)dt + \sigma \cdot dW_t$$

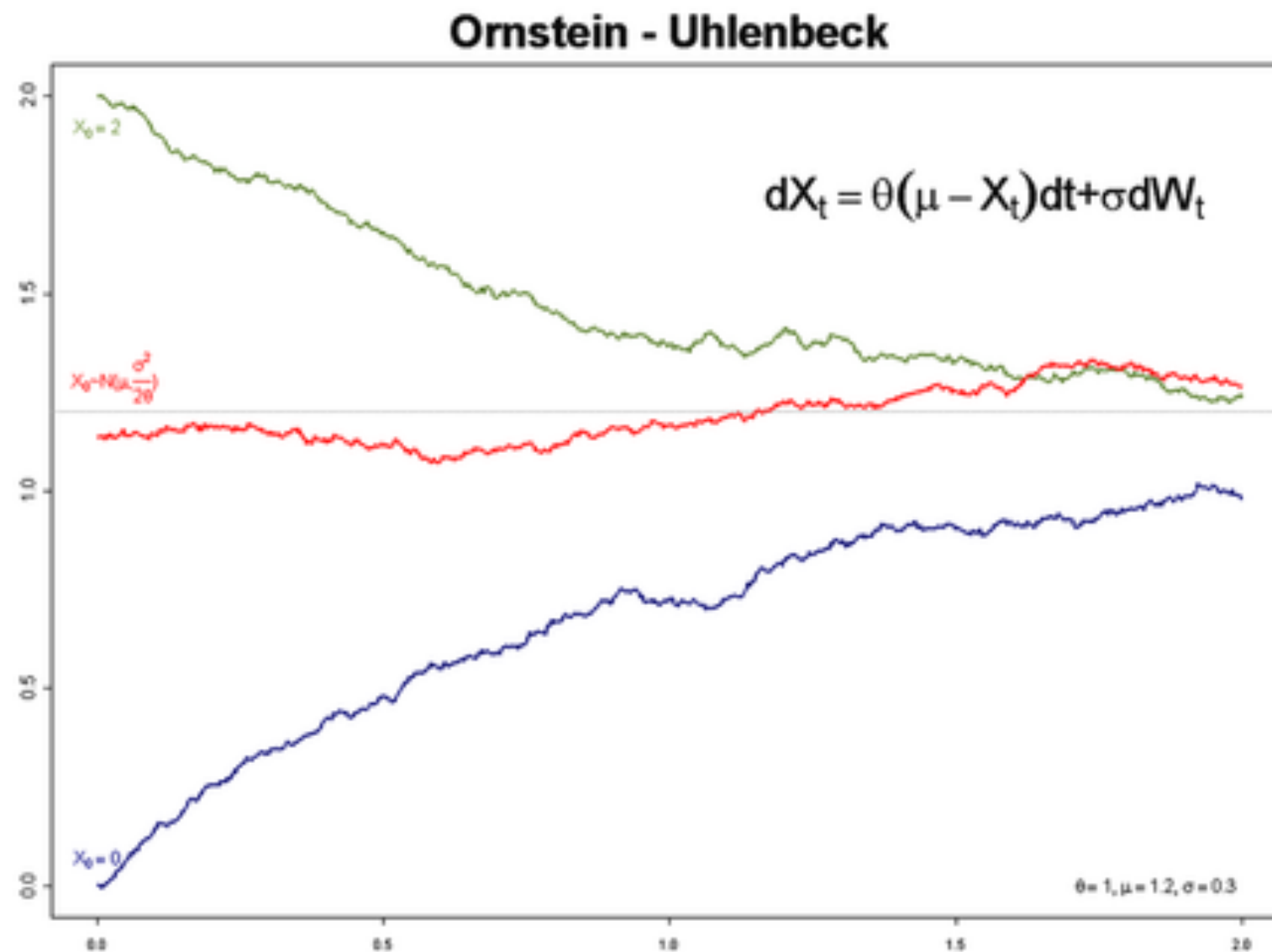
Brownian motion

- ▶ Discrete-time approximation of OU:

$$X_{t+1} - X_t = \theta(\mu - X_t)\Delta t + \sigma \cdot \Delta W_t$$

i.i.d. normal random variables $\sim \mathcal{N}(0, \Delta t)$

Example of OU Process



(Same OU process with 3 different initial conditions)

How about a sequence of i.i.d. Gaussian random variables?

(T3) Target Networks

- **Idea**: Use separate *target networks* ($\bar{\pi}_\theta$ for actor, $\bar{Q}_w$ for critic) that are updated only periodically
- For DDPG, the critic update with target networks

$$\Delta w_k \leftarrow \Delta w_k + \alpha_w \left(r_t + \gamma \overset{\text{target}}{\bar{Q}_{w_k}}(s_{t+1}, \bar{\pi}_\theta(s_{t+1})) - \overset{\text{update}}{Q_{w_k}}(s_t, a_t) \right) \nabla_w Q_w(s_t, a_t) \big|_{w=w_k}$$

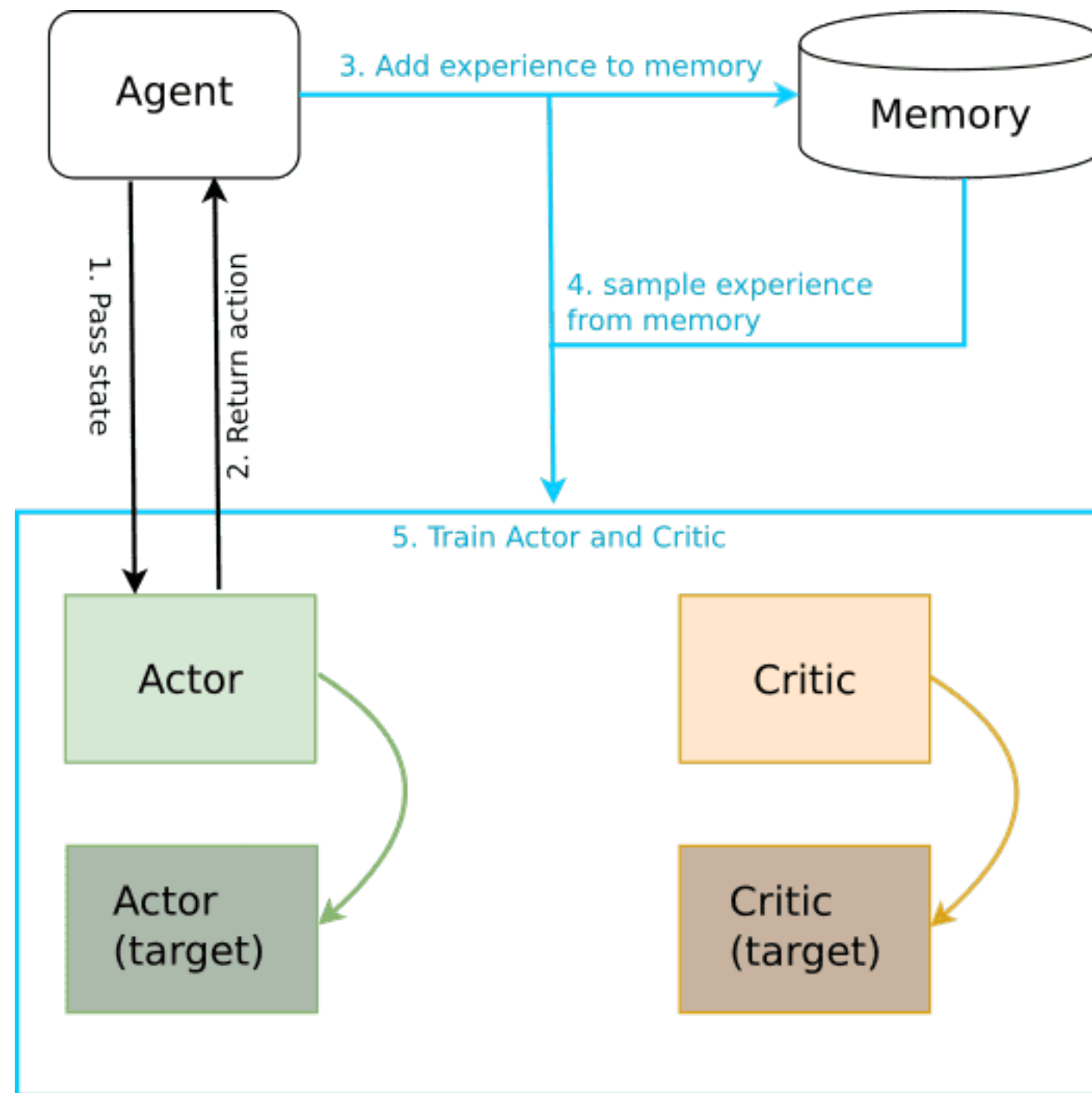
- Similar to value iteration:

$$\overset{\text{update}}{V(s)} \leftarrow \max_a R(s, a) + \gamma \sum_{s'} P(s' | s, a) \overset{\text{target}}{\bar{V}}(s)$$

- **Purpose**: Mitigate divergence

(Slide Credit: Pascal Poupart)

DDPG Architecture



Pseudo Code of DDPG Algorithm

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .

Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^\mu$

Initialize replay buffer R

for episode = 1, M **do**

Initialize a random process $\mathcal{N}$ for action exploration

Receive initial observation state s_1

for $t = 1, T$ **do**

Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise

Execute action a_t and observe reward r_t and observe new state s_{t+1}

Store transition (s_t, a_t, r_t, s_{t+1}) in R

Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R

Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$

Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$

Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

end for

end for

2 evaluation networks
and 2 target networks

action drawn from a deterministic
policy with exploration

experience replay

Update actor and critic

→ This can be viewed as
the gradient of Q w.r.t. θ

Update target networks
(small τ for stability)

Twin Delayed DDPG (TD3)

Fujimoto et al., “Addressing Function Approximation Error in Actor-Critic Methods,”
ICML 2018

Overestimation Bias in DDPG

- **Overestimation bias:** Estimated Q values are larger than the true ones

$$\mathbb{E}_s[Q_w(s, \pi_\theta(s))] \geq \mathbb{E}_s[Q^{\pi_\theta}(s, \pi_\theta(s))]$$

- **Question:** How to empirically measure such bias?
- **Empirical evidence:**

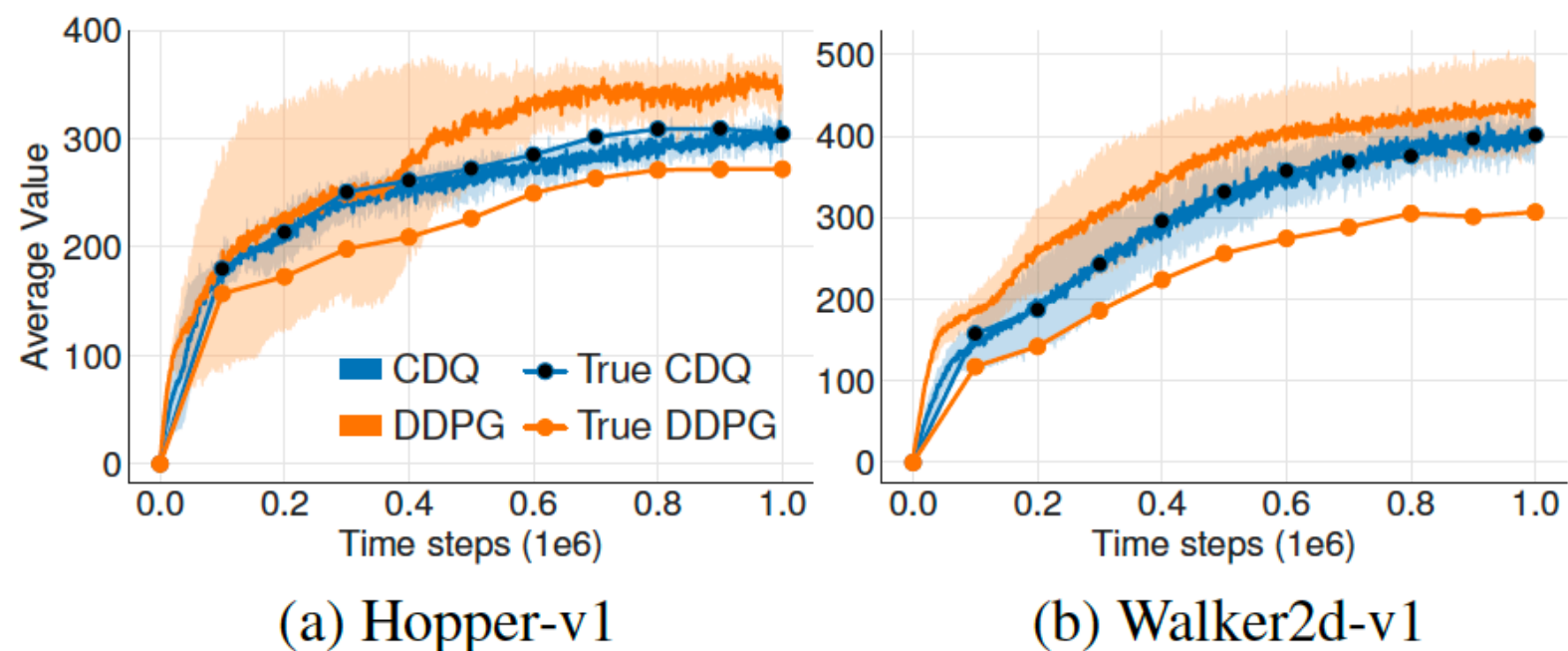


Figure 1. Measuring overestimation bias in the value estimates of DDPG and our proposed method, Clipped Double Q-learning (CDQ), on MuJoCo environments over 1 million time steps.

Overestimation Bias: A Motivating Example

- **Example:** Consider a 1-state MDP with 2 actions



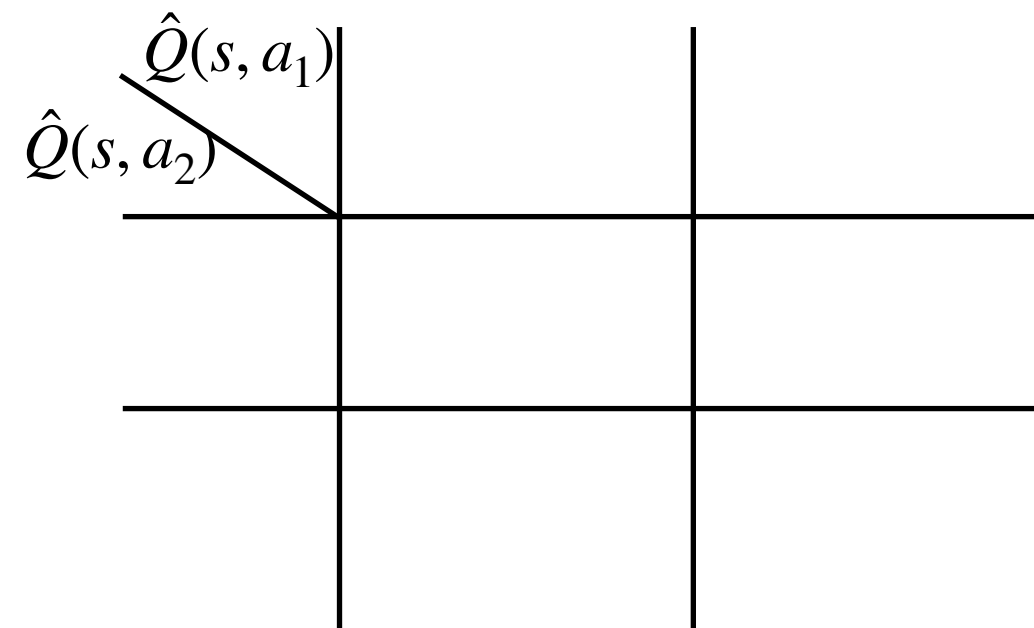
- Let $\hat{Q}(s, a_1)$, $\hat{Q}(s, a_2)$ be unbiased estimators (each based on 1 sample)

- **An Interesting Fact:**

$$\underbrace{\mathbb{E}[\max\{\hat{Q}(s, a_1), \hat{Q}(s, a_2)\}]}_{=:M} > \max\{\mathbb{E}[\hat{Q}(s, a_1)], \mathbb{E}[\hat{Q}(s, a_2)]\}$$

$$\max\{\mathbb{E}[\hat{Q}(s, a_1)], \mathbb{E}[\hat{Q}(s, a_2)]\} =$$

$$\mathbb{E}[\max\{\hat{Q}(s, a_1), \hat{Q}(s, a_2)\}] =$$



“Clipped Double-Q” (CDQ) in TD3

- ▶ **Heuristic 1:** Use two Q networks and take the minimum

DDPG’s TD-target: $y = r + \gamma Q_w(s', \pi_\theta(s'))$

TD3’s TD-target: $y = r + \gamma \min_{i=1,2} Q_{w_i}(s', \pi_\theta(s'))$

- ▶ **Heuristic 2:** Adding noise for a smooth Q

TD3’s clipped noise: $y = r + \gamma \min_{i=1,2} Q_{w_i}(s', \pi_\theta(s')) + \epsilon$

$$\epsilon \sim \text{Clip}(N(0, \sigma^2), \epsilon_{\min}, \epsilon_{\max})$$

Mitigating Overestimation Bias via CDQ: An Example

- **Example:** Consider a 1-state MDP with 2 actions

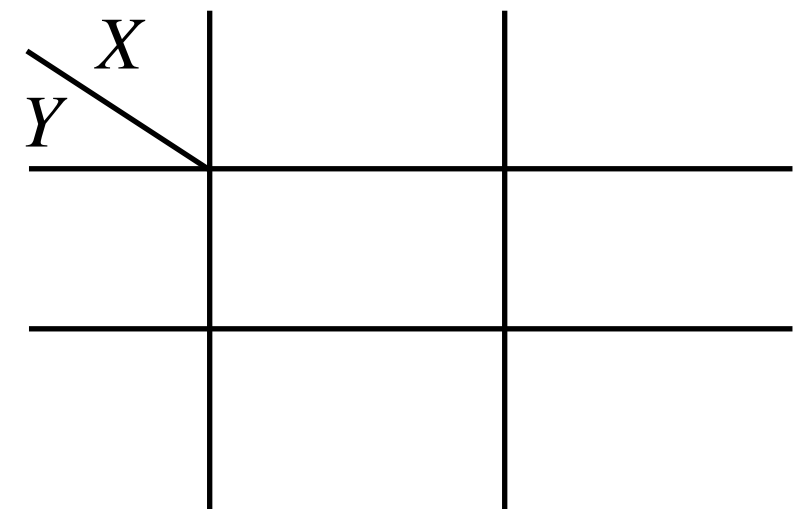


CDQ uses twin Q functions:

- Let $\hat{Q}_a(s, a_1), \hat{Q}_a(s, a_2)$ be the first unbiased Q estimate
- Let $\hat{Q}_b(s, a_1), \hat{Q}_b(s, a_2)$ be the second unbiased Q estimate

$$\max \{ \mathbb{E}[\hat{Q}(s, a_1)], \mathbb{E}[\hat{Q}(s, a_2)] \} =$$

$$\mathbb{E} \left[\max \left\{ \underbrace{\min \{ \hat{Q}_a(s, a_1), \hat{Q}_b(s, a_1) \}}_{=: X}, \underbrace{\min \{ \hat{Q}_a(s, a_1), \hat{Q}_b(s, a_2) \}}_{=: Y} \right\} \right] =$$



A2C/DPG/DDPG/TD3 $\approx$ Policy Iteration, But With a Slight Difference

Policy iteration:

Step 1: **Policy evaluation** for the current policy (i.e., find $Q^\pi(s, a)$)

Step 2: **Policy improvement** based on Bellman optimality equations

A2C/DPG/DDPG/TD3:

Step 1: Estimate $Q^{\pi_\theta}(s, a) \approx Q_w(s, a)$ for the current policy by TD

Step 2: Use $Q_w(s, a)$ to improve the policy by PG
(deterministic or stochastic)

► **Question:** Is $V^{\pi_\theta}(\mu)$ guaranteed to be improved in Step 2?